Ambient Polymorphism

Sam Estep

Abstract

…

1. Introduction

PostgreSQL 11.22 (2023) returns zero for the following query (National Institute of Standards and Technology, n.d.):

NumAcc4.sql

```
SELECT stddev(
  CASE
    WHEN k = 1 THEN 10000000.2
    WHEN k % 2 = 0 THEN 10000000.1
    ELSE 10000000.3
  END :: double precision
)
FROM generate_series(1, 1001) AS k;
```

Why? Up until version 12, Postgres computed standard deviation by first computing $a = \sum x$ and $b = \sum x^2$ …

float.c

```
N += 1.0;
sumX += newval;
CHECKFLOATVAL(sumX, isinf(transvalues[1]) || isinf(newval), true);
sumX2 += newval * newval;
CHECKFLOATVAL(sumX2, isinf(transvalues[2]) || isinf(newval), true);
```

… and then computing $Nb - a^2$:

float.c

```
numerator = N * sumX2 - sumX * sumX;
CHECKFLOATVAL(numerator, isinf(sumX2) || isinf(sumX), true);

/* Watch out for roundoff error producing a negative numerator */
if (numerator <= 0.0)
    PG_RETURN_FLOAT8(0.0);
```

This bug was eventually discovered by a third-party domain expert (Schubert 2018) who was searching for instances of this exact formula. This dependence on deep expertise is unfortunate, because in theory it's very simple to find these sorts of bugs; simply observe what happens when the program is run with higher precision:

> Higher precision will very often estimate a "correct" path well enough. To this end, imagine a debugger that can transform a given subprogram p, whose literal constants and variables x, y, z, … have been declared by the programmer to have precisions thought adequate at the time, into an analogous subprogram P whose corresponding literal constants and variables X, Y, Z, … are declared by the debugger to have greater precisions, preferably about twice as great. Then the debugger can execute both programs p and P simultaneously (actually interleaved) with the same input (or copies of it if the subprogram will change it) and compare their progress to see where one of the variables x, y, z, … first departs excessively from its analog X, Y, Z, … .

```moss stddev.moss
fn stddev(xs: Array[Item=Float]): Float {
  var n = 0.0;
  var sum = 0.0;
  var sum_sq = 0.0;
  for x in xs {
    n = n + 1.0;
    sum = sum + x;
    sum_sq = sum_sq + x * x;
  }
  let numerator = n * sum_sq - sum * sum;
  if numerator <= 0.0 { 0.0 } else { (numerator / (n * (n - 1.0))).sqrt() }
}
```

Listing 1: The numerically unstable one-pass algorithm for computing standard deviation, implemented in Moss.

> — Kahan (2006)

But the reason we have to *imagine* such a tool is because we would have to go out of our way to create it. The programming languages we use do not make it easy to perform such transformations unless one plan ahead and writes less straightforward code in the hope that it will pay off later. Of course, often it doesn't pay off. So often the smart thing to do is to avoid paying the cost of extra complexity by just writing the straightforward code in the first place.

This document proposes a novel programming language design which we call *ambient polymorphism*, with the thesis that **the cost of writing flexible programs can be drastically reduced by decoupling polymorphism from dataflow**. Section 2 describes the language design and current implementation. Section 3 sketches out a plan for evaluation. Section 4 situates this work within the body of relevant research literature.

2. Completed work

2.1. Language design

Listing 1 shows a translation of the Postgres `stddev` code into Moss. This example shows the basics of Moss syntax, which is fairly unremarkable. Figure 1 gives its grammar.

3. Proposed work

4. Related work

Bibliography

Kahan, William. 2006. "How Futile Are Mindless Assessments of Roundoff in Floating-Point Computation?." January 11. https://people.eecs.berkeley.edu/~wkahan/Mindless.pdf.

National Institute of Standards and Technology. n.d. "StRD Dataset NumAcc4." Accessed November 3, 2026. https://www.itl.nist.gov/div898/strd/univ/numacc4.html.

PostgreSQL Global Development Group. 2023. *PostgreSQL 11.22*. Released November 9. https://www.postgresql.org/docs/release/11.22/.

Schubert, Erich. 2018. "BUG #15307: Low numerical precision of (Co-) Variance." pgsql-bugs mailing list, August 1. https://www.postgresql.org/message-id/153313051300.1397.9594490737341194671%40wrigleys.postgresql.org.

File	F	::=	$\overline{m}\ \overline{d}$
Import	m	::=	`import` *str* $(\texttt{as}\ x)^?$ $(\texttt{use}\ *\ \mid\ \texttt{use}\ \overline{u})^?$ `;`
Use	u	::=	$x \mid x\ \texttt{as}\ x \mid .x \mid .x\ \texttt{as}\ .x$
Path	p	::=	$x \mid p\texttt{::}x$
Head	h	::=	$p \mid p.x \mid .x$
Spec	s	::=	$h \mid h\texttt{[}\overline{p\ \texttt{=}\ s}\texttt{]}$
Reference	r	::=	$p \mid p\texttt{[}\overline{p\ \texttt{=}\ s}\texttt{]}$
Type	τ	::=	$r \mid \texttt{This} \mid \texttt{()} \mid \texttt{(}\tau\texttt{)} \mid \texttt{\{}\overline{x\texttt{:}\ \tau}\texttt{\}} \mid \tau\ \texttt{|}\ \tau \mid \texttt{|}$
Declaration	d	::=	$\texttt{assume}\ \overline{h}\ \texttt{\{}\overline{d}\texttt{\}}$
			$\mid\ \texttt{type}\ x\texttt{;} \mid \texttt{type}\ x\ \texttt{=}\ \tau\texttt{;} \mid \texttt{val}\ x\texttt{:}\ \tau\ (\texttt{=}\ e)^?\texttt{;}$
			$\mid\ \texttt{tag}\ x\texttt{;} \mid \texttt{tag}\ x(\texttt{[}\overline{x}\texttt{]})^?\texttt{(}\tau\texttt{);} \mid \texttt{tag}\ x(\texttt{[}\overline{x}\texttt{]})^?\texttt{\{}\overline{x\texttt{:}\ \tau}\texttt{\};}$
			$\mid\ \texttt{fn}\ f\texttt{(}\overline{x\texttt{:}\ \tau}\texttt{)}(\texttt{:}\ \tau)^?\texttt{;} \mid \texttt{fn}\ f\texttt{(}\overline{x\texttt{:}\ \tau}\texttt{)}(\texttt{:}\ \tau)^?\ b$
			$\mid\ \texttt{context}\ x\ \texttt{=}\ \overline{s}\texttt{;}$
			$\mid\ \texttt{functor}\ x(\texttt{[}\overline{x}\texttt{]})^?\texttt{:}\ \overline{s}\ \texttt{->}\ \overline{s}\texttt{;}$
			$\mid\ \texttt{functor}\ x\texttt{:}\ \overline{s}\ \texttt{->}\ \overline{s}\ \texttt{\{}\overline{w}\texttt{\}}$
Function	f	::=	$x \mid x.x \mid .x$
Block	b	::=	$\texttt{\{}\overline{i}\ (e)^?\texttt{\}}$
Statement	i	::=	$\texttt{let}\ x(\texttt{:}\ \tau)^?\ \texttt{=}\ e\texttt{;} \mid \texttt{var}\ x(\texttt{:}\ \tau)^?\ \texttt{=}\ e\texttt{;} \mid x\ \texttt{=}\ e\texttt{;} \mid w$
			$\mid\ \texttt{while}\ e\ b \mid \texttt{loop}\ b \mid \texttt{for}\ x\ \texttt{in}\ e\ b \mid e\texttt{;} \mid e$
Bind	w	::=	$\texttt{bind}\ \overline{\beta}\texttt{;}$
Binding	β	::=	$s \mid s\ \texttt{=}\ e \mid s\ \texttt{=}\ \texttt{(}\overline{x(\texttt{:}\ \tau)^?}\texttt{)}(\texttt{:}\ \tau)^?\ \texttt{=>}\ e$
Expression	e	::=	$c \mid \texttt{match}\ e\ \texttt{\{}\overline{a}\texttt{\}} \mid \texttt{return}\ (e)^? \mid \texttt{break}$
			$\mid\ e.p\texttt{(}\overline{e}\texttt{)} \mid e.x \mid e \oplus e \mid \ominus e \mid \texttt{(}e\texttt{)} \mid \texttt{()} \mid \texttt{this} \mid b$
			$\mid\ \mathit{int} \mid \mathit{dec} \mid r \mid r\texttt{(}\overline{e}\texttt{)} \mid r\texttt{\{}\overline{x(\texttt{=}\ e)^?}\texttt{\}}$
Binary	$\oplus$	::=	`*` | `/` | `%` | `+` | `-` | `==` | `!=` | `<` | `>` | `<=` | `>=` | `<<` | `>>` | `&` | `^` | `|`
Unary	$\ominus$	::=	`-` | `!`
Conditional	c	::=	$\texttt{if}\ e\ b\ (\texttt{else}\ b \mid \texttt{else}\ c)^?$
Arm	a	::=	$\pi\ \texttt{=>}\ e\texttt{,} \mid \pi\ \texttt{=>}\ b\ (\texttt{,})^?$
Pattern	π	::=	$\texttt{_} \mid p \mid p\texttt{(}\pi\texttt{)} \mid \texttt{\{}\overline{x(\texttt{=}\ \pi)^?}\texttt{\}} \mid p\texttt{\{}\overline{x(\texttt{=}\ \pi)^?}\texttt{\}}$

Figure 1: The grammar of Moss. Here x ranges over names, and *str*, *int* and *dec* over string, integer and decimal literals; $(X)^?$ marks an optional X, and $\overline{X}$ a possibly empty sequence of X, separated by commas (with an optional trailing comma) unless X is an import, declaration, statement, bind or arm. A bare `|` is the uninhabited type. An expression statement may omit its `;` only if it ends in `}`.